

Regex - colinha básica

Uma expressão regular (regex) é um **padrão** que descreve um conjunto de textos. Duas regras valem para tudo abaixo:

- 1. Sem âncoras, o padrão casa em **qualquer posição** do texto - e pode casar várias vezes. [0-9]{3} em 2026 casa «202».
- 2. Todo caractere casa **ele mesmo**, exceto os metacaracteres . [] ^ \$ * + ? { } () | \ - para casar um deles ao pé da letra, ponha \ na frente: \. \. \$ \. (\. \.

Caracteres

Símbolo	Significa	Exemplo	Casa
.	um caractere qualquer	a.c	abc, a-c, a c
\.	um ponto literal	a\.c	a.c
[abc]	um caractere do conjunto	[gr]ato	gato, rato
[a-z]	um caractere do intervalo	[a-z][0-9]	b2
[a-zA-Z0-9]	intervalos convivem na mesma classe	[A-Z0-9]	Q, 7
[^abc]	um caractere FORA do conjunto	[^0-9]	a, %, espaço
\d \w \s	atalhos: dígito, letra/dígito/_ , espaço	\d\d	42

Dentro de [] os metacaracteres perdem o poder: [.] é um ponto literal. O hífen vai no fim ([0-9-]) e o ^ só nega se for o primeiro.

Âncoras

Não casam caracteres, casam **posições**.

Símbolo	Significa	Exemplo	Casa	Não casa
^	início do texto	^casa	casarão	descasar
\$	fim do texto	fim\$	o fim	fim da linha
^...\$	o texto é exatamente isto	^cat\$	cat	cats, gato
\b	fronteira de palavra	\bcata\b	cat	concatenar

Quantificadores

Aplicam-se ao item **imediatamente anterior**: um caractere, uma classe [] ou um grupo (). ab+ é um a e vários b; (ab)+ é ab repetido.

Símbolo	Repetições	Exemplo	Casa	Não casa
*	zero ou mais	ba*nana	bnana, banana, baaanana	bxnana
+	uma ou mais	ba+nana	banana, baaanana	bnana
?	zero ou uma (opcional)	casas?	casa, casas	casinha
{n}	exatamente n	[0-9]{3}	123	12
{n,m}	de n a m	a{2,4}	aa, aaa, aaaa	a
{n,}	n ou mais	[0-9]{2,}	12, 12345	1

* e + são **gulosos**: pegam o maior trecho possível. * e ? também casam **zero** vezes - [0-9]* é satisfeito por um texto sem dígito nenhum.

Alternância e grupos

Símbolo	Significa	Exemplo	Casa	Não casa
	ou	gato rato	gato, rato	pato
()	grupo	(g r)ato	gato, rato	pato
()+	grupo repetido	(ab)+	ab, abab	abb

O \ | tem a **menor** precedência: sem parênteses, divide a expressão inteira.

Regex	Lê-se
^a b\$	começa com a ou termina com b
^(a b)\$	o texto é exatamente a ou b

Exemplos num texto maior

«...» marca o trecho casado.

Texto	
A	0 gato preto subiu no telhado as 07:45 de 12/03/2026.
B	preco: R\$ 1.299,90 -- codigo AB-12345, tel (11) 98765-4321
C	disse "oi" e depois "tchau" para todos

Regex	Casa
.ato	O «gato» preto subiu...
pret[oa]s?	O gato «preto» subiu...
sub[a-z]*	...preto «subiu» no telhado...
gato telhado	O «gato» preto subiu no «telhado» as...
[0-9]+	...as «07»:«45» de «12»/«03»/«2026».
[0-9]+:[0-9]+	...as «07:45» de...
[0-9]{2}/[0-9]{2}/[0-9]{4}	...de «12/03/2026».
[A-Z]{2}-[0-9]{5}	...codigo «AB-12345», tel...
R\$ [0-9.]+,[0-9]{2}	preco: «R\$ 1.299,90» -- codigo...
\([0-9]{2}\) [0-9]{4,5}-[0-9]{4}	...tel «(11) 98765-4321»
".*"	disse «"oi" e depois "tchau"» para todos
"[^"]*"	disse «"oi"» e depois «"tchau"» para todos

Os dois últimos são a mesma pergunta - "um trecho entre aspas" - com o .* guloso engolindo tudo até a última aspa, e [^"]* parando na primeira.

Forth - colinha básica

A pilha e a notação posfixa (RPN)

Forth não tem 2 + 3. Tem 2 3 +: **primeiro os dados, depois o operador**. Tudo passa por uma **pilha**: o operador consome do topo e devolve ao topo. Não há precedência nem parênteses - a ordem em que você empilha já é a conta.

Infixa	Posfixa	Resultado
2 + 3	2 3 +	5
(2 + 3) × 4	2 3 + 4 *	20
2 + (3 × 4)	2 3 4 * +	14
100 / (4 + 1)	100 4 1 + /	20
(8 – 3) × (8 + 3)	8 3 - 8 3 + *	55

Palavra	Faz
.	tira o topo da pilha e imprime
.s	mostra a pilha inteira sem consumir: <3> 10 20 30 (topo à direita)
clearstack	esvazia a pilha - use depois de um <i>Stack underflow</i>
bye	sai do interpretador

10 20 30 + . . imprime 50 e depois 10: somou os dois do topo, imprimiu, e imprimiu o que sobrou. Pedir um valor que não existe dá **Stack underflow**.

Aritmética com inteiros

O valor empilhado **primeiro** é o da esquerda da conta: 10 3 - é 10 – 3. A divisão é **inteira** (trunca): 7 2 / dá 3, não 3.5.

Palavra	Efeito na pilha	Exemplo	Imprime
+ - * /	(a b -- r)	10 3 / .	3
MOD	(a b -- resto)	10 3 MOD .	1
/MOD	(a b -- resto quociente)	10 3 /MOD .	3 1
*/	(a b c -- a×b/c) sem estourar o produto	100 3 4 */ .	75
ABS NEGATE	(a -- a) (a -- -a)	-7 ABS .	7
1+ 1-	(a -- a+1) (a -- a–1)	7 1+ .	8
2* 2/	(a -- 2a) (a -- a/2)	7 2/ .	3
MIN MAX	(a b -- menor) (a b -- maior)	3 9 MAX .	9

Manipulação da pilha

Não há variáveis para os valores intermediários: são estas palavras que reorganizam a pilha. A notação (antes -- depois) lê-se com o **topo à direita** dos dois lados.

Palavra	Efeito	Faz	Antes	Depois
DUP	(a -- a a)	duplica o topo	1 2 3	1 2 3 3
DROP	(a --)	joga fora o topo	1 2 3	1 2
SWAP	(a b -- b a)	troca os dois do topo	1 2 3	1 3 2
OVER	(a b -- a b a)	copia o segundo para o topo	1 2 3	1 2 3 2
ROT	(a b c -- b c a)	traz o terceiro para o topo	1 2 3	2 3 1
NIP	(a b -- b)	remove o segundo	1 2 3	1 3
TUCK	(a b -- b a b)	copia o topo para baixo do segundo	1 2 3	1 3 2 3
2DUP	(a b -- a b a b)	duplica o par do topo	1 2	1 2 1 2
2DROP	(a b --)	joga fora o par do topo	1 2 3 4	1 2
2SWAP	(a b c d -- c d a b)	troca os dois pares	1 2 3 4	3 4 1 2
DEPTH	(-- n)	empilha quantos itens há	1 2 3	1 2 3 3

ROT mexe só nos três do topo; do quarto para baixo nada é tocado.

Receitas

Objetivo	Código	Resultado
a² sem repetir a	5 DUP * .	25
a³	17 DUP DUP * * .	4913
a² + b² com 3 e 4 na pilha	3 4 DUP * SWAP DUP * + .	25
inverter 1 2 3 em 3 2 1	1 2 3 SWAP ROT .s	<3> 3 2 1
NIP com o que já existe	SWAP DROP	
TUCK com o que já existe	SWAP OVER	
2DUP com o que já existe	OVER OVER	
ROT ROT	traz o topo para o terceiro lugar	1 2 3 → 3 1 2

Lisp / Racket - colinha básica

Sintaxe: só existe a s-expression

Uma sexp é um **átomo** (42, 3.14, "olá", soma) ou uma **lista** de sexps entre parênteses. Uma chamada é uma lista cujo **primeiro** elemento é a função: notação **prefixa**, sem vírgulas, sem precedência - o aninhamento dá a ordem. Todo parêntese aberto tem que fechar.

Expressão	Resultado	
(+ 2 3)	5	
(* 2 3 4)	24	dois ou mais argumentos
(+ 2 (* 3 4))	14	o mais interno primeiro
(- 5)	-5	
(/ 10 3)	10/3	fração exata , não trunca
(/ 10.0 3)	3.3333333333333335	
(quotient 10 3) (remainder 10 3)	3 1	divisão inteira e resto
(expt 2 10) (sqrt 16)	1024 4	
(max 3 9) (min 3 9)	9 3	

Pares e listas

Tudo é feito de **células cons**: um par [car | cdr] . Uma lista é uma cadeia de cons que termina em '() , a lista vazia.

Expressão	Resultado	
(cons 4 5)	'(4 . 5)	par; o ponto = não é lista

Expressão	Resultado	
(car (cons 4 5)) (cdr (cons 4 5))	4 5	
(cons 10 (cons 100 '()))	'(10 100)	[10 *]->[100 *]->'()
'(10 100 1000) (list 10 100 1000)	'(10 100 1000)	mesma lista; list avalia os argumentos
(car '(10 100 1000))	10	a cabeça
(cdr '(10 100 1000))	'(100 1000)	o resto - sempre uma lista
(cons 1 '(2 3))	'(1 2 3)	põe na frente
(cons 1 (cons 2 3))	'(1 2 . 3)	não termina em '(): não é lista
(null? '()) (null? '(1))	#t #f	
(list? '()) (pair? '())	#t #f	
(pair? (cons 4 5)) (list? (cons 4 5))	#t #f	

quote, eval, apply

Toda lista é candidata a programa (homoiconicidade). 'x é açúcar para (quote x): **desliga** a avaliação e devolve a sexp como dado. eval faz o caminho inverso. apply chama uma função com os argumentos vindos numa lista.

Expressão	Resultado
(+ 2 3)	5
'(+ 2 3)	'(+ 2 3)
(eval '(+ 2 3))	5
(apply + '(1 2 3 4))	10 - o mesmo que (+ 1 2 3 4)
(apply max '(3 7 2 9 1))	9

define

(define nome valor) para variável; (define (nome parâmetros) corpo) para função. O corpo é **uma expressão**, cujo valor é o retorno.

(define (maior a b) (if (> a b) a b))	; nome: maior parâmetros: a b ; teste a ; então b)) ; senão
(maior 7 10)	; 10

É açúcar para (define maior (lambda (a b) (if (> a b) a b))) : uma função é um valor como outro qualquer, produzido por lambda.

Condicionais, comparações, lógicos

if é uma **expressão** (produz valor) com três partes obrigatórias: (if teste então senão). O **único** valor falso é #f - 0, "" e '() contam como verdadeiro.

Expressão	Resultado
(if (> 3 10) (+ 4 10) (- 4 10))	-6
(if 0 "v" "f") (if '() "v" "f")	"v" "v"
(= 3 3) (< 3 10) (>= 10 3)	#t - só para números
(equal? '(1 2 3) '(1 2 3)) (equal? "oi" "oi")	#t - compara qualquer coisa
(and (> 10 6) (= 2 2)) (or (> 3 10) (= 5 5)) (not (> 3 10))	#t #t #t
(and 1 2 3) (or #f #f 5)	3 5 - devolvem o valor , não só #t
(number? 3) (symbol? 'a) (string? "x") (list? '(1))	#t - predicados de tipo

cond encadeia vários testes; else fecha:

(cond [(< n 0) "negativo"] [(= n 0) "zero"] [else "positivo"])
--

Recursão sobre listas - colinha básica

O esqueleto (é sempre o mesmo)

Toda função recursiva sobre uma lista tem três ingredientes: a lista é um parâmetro; o **primeiro teste** é (null? l); no caso geral aparecem a cabeça (car l) e uma chamada recursiva sobre a cauda (cdr l).

```
(define (f l)
  (if (null? l)
      <caso base>
      (<op> (car l) (f (cdr l))))) ; cabeça + resto
```

Sem o teste (null? l) a recursão não para: chega em (cdr '()) e quebra.

O pulo do gato

A máquina **não vê a lista inteira** - só a cabeça. O resto é um borrão:

```
Humano: '(4 0 -2 3 9 6 20 -4 7)
Máquina: '(4 <bla bla bla...>)
```

Não tente enxergar através do borrão nem simular todas as chamadas. Há só **dois tijolinhos** na mesa:

Tijolinho	O que é
(car l)	a cabeça - o que você vê de verdade
(f (cdr l))	o resto já resolvido, por mágica : trate como um dado pronto

Sua única tarefa é escolher o **operador** que combina os dois. Como a recursão chegou ao resultado da cauda não é problema seu.

A tabela: o tipo do retorno decide o resto

O caso base é o **elemento neutro** do operador do caso geral.

Tipo de retorno	Caso base	Operador do caso geral
número	0, 1, ...	+, add1, *, ...
lista	'()	cons
booleano	#f ("existe algum?") / #t ("todos?")	or / and, not

Numa lista vazia não existe nenhum (#f) e todos satisfazem, por vacuidade (#t).

Quatro funções, uma por tipo

```
; número: somar os elementos
; (somar '(4 6 0 2)) → 12
(define (somar l)
  (if (null? l)
      0
      (+ (car l) (somar (cdr l)))))

; lista: o dobro de cada elemento
; (dobrar '(6 7 -3)) → '(12 14 -6)
(define (dobrar l)
  (if (null? l)
      '()
      (cons (* 2 (car l)) (dobrar (cdr l)))))

; booleano: x está na lista?
; (pertence? 3 '(0 3 1)) → #t
(define (pertence? x l)
  (if (null? l)
      #f
      (if (equal? x (car l))
          #t
          (pertence? x (cdr l)))))

; função como parâmetro: só os que passam no teste
; (my-filter even? '(2 3 4 6 7)) → '(2 4 6)
(define (my-filter p l)
  (if (null? l)
      '()
      (if (p (car l))
          (cons (car l) (my-filter p (cdr l)))
          (my-filter p (cdr l)))))
```

Use cons, não list, no caso geral: (list x (dobrar (cdr l))) cria uma lista de dois elementos com a cauda inteira aninhada dentro.

Para reconhecer um problema novo

Pergunta	Opções
Qual o tipo do retorno?	número, lista, booleano → tabela acima
Quantas vezes a cabeça entra no resultado?	uma (dobrar), zero ou uma conforme teste (my-filter), duas
Quanto a recursão anda?	um elemento (comum), dois, um em cada lista
Quantos casos base?	um ('()) ou dois (lista unitária, segunda lista, contador)
A operação é fixa ou parâmetro?	fixa: dobrar, pertence? - parâmetro: my-map, existe?, my-filter

Escrita à mão	Generalizada	Nativa em Racket	Faz
dobrar	my-map	map	transforma cada elemento
pertence?	existe?	ormap	pergunta "algum?"
-	todos?	andmap	pergunta "todos?"
remove-todos	my-filter	filter	seleciona alguns